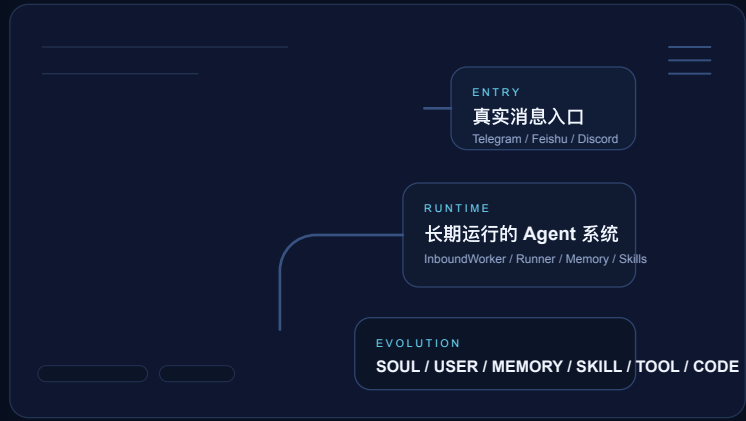
GOFENIX / NEXAGENT

NexAgent

从“养龙虾”到
长期运行的 Agent 系统

为什么在 OpenClaw 之后， 我们还要再做一个自己的 agent?



QUESTION

既然已经有了 OpenClaw，为什么还要再做一个 NexAgent？

OPENCLAW 打开的门

Agent 已经开始像“系统”

- 会写代码
- 会串流程
- 会调用工具
- 会长出意料之外的能力

以前更像预制软件。现在开始出现“现场组合能力”的迹象。

真正的问题在后面

如果它要长期存在，该怎么活着?

- 怎么长期在线
- 怎么进入真实聊天环境
- 怎么组织会话和记忆
- 学到的新东西放在哪里
- 未来怎么稳定地继续长

SHIFT

问题已经从

“怎么做一个 agent demo”

变成了

“怎么做一个长期运行的 agent 系统”

为什么要自己做

自己做一个 agent 的意义，在于真正从使用者切换到创造者。

1. 你会真正区分 prompt 问题和系统问题

2. 你会亲手踩到多通道、会话、记忆、调度这些坑

3. 你会被迫回答：这个 agent 的底层结构到底应该是什么

POSITION

NexAgent 的出发点，是把“养龙虾”带来的直觉继续往系统设计的方向推进一步。

继续追问

真正把问题逼出来的

不是“再做一个 demo”，而是下面这三个问题。

1. 记忆到底是什么？

有价值的往往不是“记得我是谁”，而是“记得我是怎么做事的”。

2. agent 的主动性从哪里来？

很多时候不是玄学规划，而是调度、定时器和后台执行结构。

3. 新学到的东西该放在哪里？

如果最后都堆进 prompt 或 memory，系统很快就会糊成一团。

PART 1

为什么是 Elixir / OTP

如果只是单轮执行器

- 调一次模型
- 做几次 tool calling
- 返回一个结果

技术选型空间很大。

如果是长期在线系统

- 多聊天通道接入
- 多会话并行
- 后台任务和定时任务
- 失败恢复
- 热更新
- 未来可能还有自我升级

运行时能力会直接变成产品能力。

为什么是 Elixir / OTP

因为我们要承载的不是一次性的模型调用，而是一套长期活着的 agent runtime。

RECOVERY
监督树
把失败当成常态处理

BOUNDARY
进程隔离
会话 / 任务天然有边界

CONCURRENCY
并发模型
多入口 / 多任务更自然

UPGRADE
热代码加载
为运行时升级留下空间

WHY IT FITS
长期在线、可恢复、可拆分、可热更新
这不是“工程师喜欢什么语言”的问题，而是 agent runtime 需要什么运行模型的问题。

PART 2

NexAgent 里面到底有什么

一个长期 Agent 系统的五层结构

入口、运行、状态、后台和进化不是散落的功能，而是一个连续的 runtime。

ENTRY
入口层
Gateway + Channels

RUNTIME
运行层
InboundWorker + Runner

STATE + CAPABILITY
状态与能力层
SessionManager / Memory / Skil

BACKGROUND
后台执行层
Cron / Subagent

EVOLUTION
进化层
系统不会把所有变化都混成一个模糊的 memory，而是把变化送进不同层级。

SOUL | USER | MEMORY | SKILL | TOOL | CODE

入口层

Gateway + Channels

让 agent 真正接住持续流动的消息流。

运行层

InboundWorker + Runner

把消息、安全调度、tool calling 和 agent loop 组织成稳定过程。

状态与能力层

SessionManager / Memory Tool.Registry / Skills

让系统具备长期会话、长期事实、方法沉淀和能力扩展。

后台执行层

Cron / Subagent

让系统获得主动执行和后台拆分执行能力。

核心模块

入口

Gateway

统一编排聊天入口。

调度中枢

Runner

上下文、LLM loop、tool calling 的中枢。

入口到运行

InboundWorker

消息路由、会话调度、排队。

长期状态

Memory

把用户信息和环境事实分层保存。

能力扩展

Tool.Registry / Skills

把能力和方法拆开，避免堆成一团。

升级路径

CodeUpgrade / UpgradeManager

给 code-level evolution 一条受控路径。

PART 3

进化如果不分层，系统很快就会变形

错误路径

所有变化都堆在一起

- 今天改一点 prompt
- 明天记一条 memory
- 后天补一个脚本
- 接着再塞一个工具

结果：难判断、难复用、难升级。

正确路径

先决定变化该落在哪一层

- 事实进入 `USER / MEMORY`
- 方法沉淀成 `SKILL`
- 稳定能力升级成 `TOOL`
- 只有最后才进入 `CODE`

系统会保持可理解、可迭代。

NexAgent 的六层进化结构

变化优先落在更轻、更高的层。先记住事实，再沉淀方法，再扩展能力，最后才进入代码。

更高 / 更轻

SOUL	身份、价值观、长期行为原则
USER	用户画像、协作偏好
MEMORY	环境、项目和上下文中的长期事实
SKILL	稳定的方法和流程，能够复用的程序性知识
TOOL	确定性的可执行能力
CODE	系统内部实现的变化，最后才进入这一层

更低 / 更重

分流原则

优先级

变化优先落在 **更轻、更高** 的层。

1. 能进入 `USER / MEMORY`，就先不要急着写 `SKILL`

2. 能沉淀成 `SKILL`，就先不要急着做 `TOOL`

3. 能做成 `TOOL`，就先不要急着改 `CODE`

进入 CODE 层时怎么做

系统允许 code-level evolution，但它必须是一条有入口、有过程、有边界的路径。

INSPECT
reflect
看源码 / 历史 / 差异

SUBMIT
upgrade_code
提交新的模块实现

VALIDATE
CodeUpgrade
备份 / 验证 / 编译 / 加载

APPLY
UpgradeManager
热更新 / 版本保存 / 回滚保护

BOUNDARY
系统不回避代码升级，但它拒绝无边界地乱改

PART 4

为什么这件事值得做

我们真正关心的

Agent 能不能从一个好用的 demo，变成一个长期在线的系统?

Agent 能不能真正进入聊天环境，成为日常工作流的一部分?

Agent 能不能不只记住标签和片段，而是逐渐学会方法、沉淀方法、复用方法?

Agent 的能力扩展，能不能有一个不会越来越乱的结构?

CONCLUSION

未来更有价值的，不是一个平均意义上“什么都会一点”的 agent。

更有价值的，是一个专属系统。

它能够理解具体用户。

它服务具体工作流。

它在真实环境里持续进化。

CLOSING

NexAgent

它未必已经给出最终答案。

但它在认真把这些问题落到一个 真实可运行的系统 里。

github.com/gofenix/nex-agent